

Нейромережа Ельмана

Нейромережа Ельмана – це рекурентна нейронна мережа (RNN), яка має контекстний шар. Це дозволяє їй запам'ятовувати попередні стани та аналізувати послідовності. Вона використовується для розпізнавання мовлення, прогнозування рядів, керування роботами тощо.

Структура нейромережі Ельмана

1. **Вхідний шар** (вхідні нейрони).
2. **Прихований шар** (звичайні нейрони, що передають сигнали далі).
3. **Контекстний шар** (копія виходів прихованого шару, яка зберігається на один такт часу).
4. **Вихідний шар** (формує остаточний результат).

Контекстний шар додає "пам'ять", зберігаючи інформацію про попередні стани.

```
pip install tensorflow
```

Код нейромережі Ельмана в Python (TensorFlow/Keras)

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

# Генеруємо випадкові послідовності як навчальні дані
X_train = np.random.rand(1000, 10, 1) # 1000 прикладів,
10 кроків, 1 ознака
y_train = np.sum(X_train, axis=1) # Вихід - сума всіх
значень у послідовності

# Створюємо модель нейромережі Ельмана
model = Sequential([
```

```

        SimpleRNN(10, activation="tanh",
return_sequences=False, input_shape=(10, 1)), #
Контекстний шар
        Dense(1, activation="linear") # Вихідний шар
    ])

# Компілюємо модель
model.compile(optimizer="adam", loss="mse")

# Навчання моделі
model.fit(X_train, y_train, epochs=20, batch_size=16)

# Перевірка на нових даних
X_test = np.random.rand(10, 10, 1)
y_pred = model.predict(X_test)

print("Передбачені значення:", y_pred.flatten())

```

Що тут відбувається?

SimpleRNN – реалізує **контекстний шар** для запам'ятовування попередніх станів.

return_sequences=False – видає один вихід (останній стан).

Dense(1, activation="linear") – видає числовий результат (регресія).

Adam + MSE – оптимізатор і функція втрат для стабільного навчання.

Застосування нейромережі Ельмана

- Прогнозування часових рядів
- Аналіз мовлення та тексту
- Керування динамічними системами

pip install torch

Реалізація на PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# Гіперпараметри
input_size = 1      # Кількість входів
hidden_size = 5     # Розмір прихованого шару
output_size = 1     # Кількість виходів
epochs = 1000
learning_rate = 0.01

# Створюємо навчальні дані (штучний часовий ряд)
x_train = np.linspace(0, 10, 100)
y_train = np.sin(x_train)

# Перетворюємо у тензори PyTorch
x_train = torch.tensor(x_train,
dtype=torch.float32).view(-1, 1)
y_train = torch.tensor(y_train,
dtype=torch.float32).view(-1, 1)

# Нейромережа Ельмана
class ElmanNN(nn.Module):
    def __init__(self, input_size, hidden_size,
output_size):
        super(ElmanNN, self).__init__()
```

```

        self.hidden_size = hidden_size
        self.i2h = nn.Linear(input_size + hidden_size,
hidden_size)  # Вхід -> Прихований шар
        self.h2o = nn.Linear(hidden_size, output_size)  #
Прихований -> Вихід

    def forward(self, x, hidden):
        combined = torch.cat((x, hidden), 1)  #
Об'єднуємо поточний вхід та прихований стан
        hidden = torch.tanh(self.i2h(combined))
        output = self.h2o(hidden)
        return output, hidden

# Ініціалізація мережі
model = ElmanNN(input_size, hidden_size, output_size)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters()),
lr=learning_rate)

# Навчання
hidden = torch.zeros(1, hidden_size)  # Початковий
прихований стан
for epoch in range(epochs):
    optimizer.zero_grad()
    loss = 0
    for i in range(len(x_train)):
        output, hidden = model(x_train[i].view(1, -1),
hidden)
        loss += criterion(output, y_train[i].view(1, -1))
    loss.backward()
    optimizer.step()

```

```

        if epoch % 100 == 0:
            print(f'Epoch {epoch}, Loss: {loss.item():.4f}')

# Перевірка результату
preds = []
hidden = torch.zeros(1, hidden_size)
for i in range(len(x_train)):
    output, hidden = model(x_train[i].view(1, -1),
hidden)
    preds.append(output.item())

# Візуалізація
import matplotlib.pyplot as plt
plt.plot(x_train.numpy(), y_train.numpy(), label="Реальні
дані", color="blue")
plt.plot(x_train.numpy(), preds, label="Прогноз мережі
Ельмана", color="red", linestyle="dashed")
plt.legend()
plt.show()

```

Реалізація на NumPy (без глибокого навчання)

Можна реалізувати спрощену **нейромережу Ельмана на NumPy**:

```

import numpy as np

# Гіперпараметри
input_size = 1
hidden_size = 5
output_size = 1
learning_rate = 0.01
epochs = 1000

# Ініціалізація ваг
W_xh = np.random.randn(hidden_size, input_size)
W_hh = np.random.randn(hidden_size, hidden_size)
W_hy = np.random.randn(output_size, hidden_size)

```

```

# Навчальні дані
x_train = np.linspace(0, 10, 100).reshape(-1, 1)
y_train = np.sin(x_train)

# Початковий прихований стан
hidden = np.zeros((hidden_size, 1))

# Функція активації (гіперболічний тангенс)
def tanh(x):
    return np.tanh(x)

# Навчання
for epoch in range(epochs):
    total_loss = 0
    for i in range(len(x_train)):
        x = x_train[i].reshape(-1, 1) # Вхід
        target = y_train[i].reshape(-1, 1) # Реальна відповідь

        # Прямий прохід
        hidden = tanh(np.dot(W_xh, x) + np.dot(W_hh, hidden))
        output = np.dot(W_hy, hidden)

        # Помилка
        loss = np.mean((output - target) ** 2)
        total_loss += loss

        # Зворотне поширення (градієнтний спуск)
        d_output = 2 * (output - target)
        d_W_hy = np.dot(d_output, hidden.T)
        d_hidden = np.dot(W_hy.T, d_output) * (1 - hidden ** 2)
        d_W_xh = np.dot(d_hidden, x.T)
        d_W_hh = np.dot(d_hidden, hidden.T)

        # Оновлення ваг
        W_hy -= learning_rate * d_W_hy
        W_xh -= learning_rate * d_W_xh
        W_hh -= learning_rate * d_W_hh

    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Loss: {total_loss:.4f}")

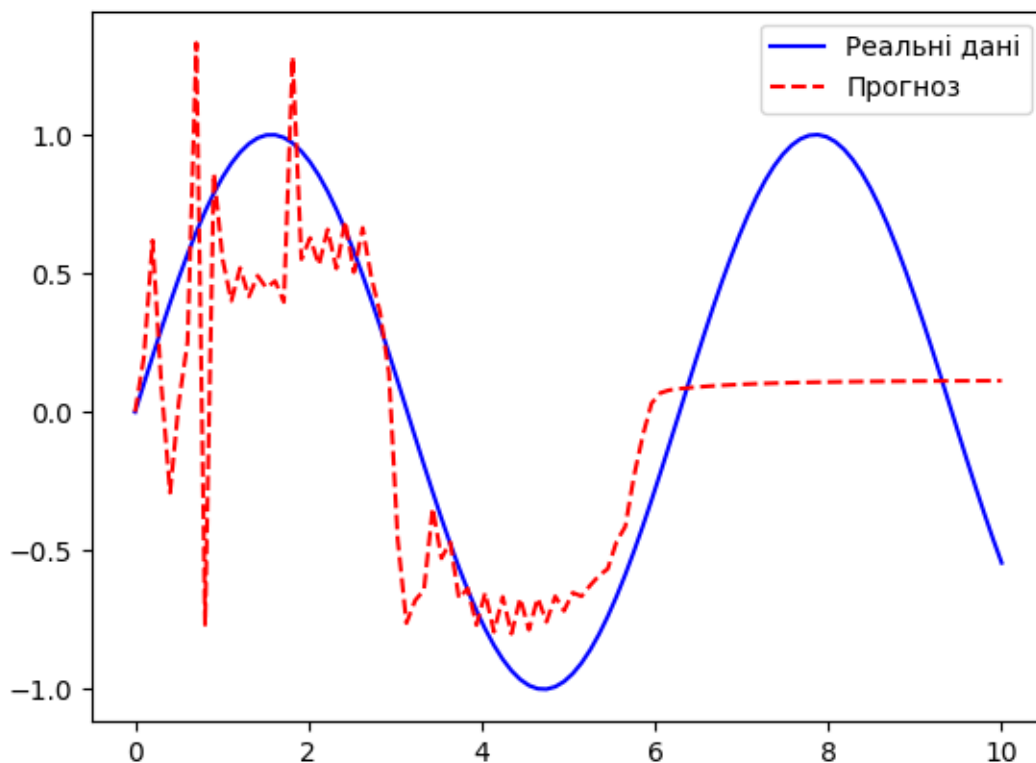
# Прогнозування
preds = []
hidden = np.zeros((hidden_size, 1))
for i in range(len(x_train)):
    x = x_train[i].reshape(-1, 1)
    hidden = tanh(np.dot(W_xh, x) + np.dot(W_hh, hidden))
    output = np.dot(W_hy, hidden)
    preds.append(output.item())

# Графік
import matplotlib.pyplot as plt
plt.plot(x_train, y_train, label="Реальні дані", color="blue")

```

```
plt.plot(x_train, preds, label="Прогноз", color="red", linestyle="dashed")
plt.legend()
plt.show()
```

```
Epoch 0, Loss: 95.4002
Epoch 100, Loss: 17.4708
Epoch 200, Loss: 10.4847
Epoch 300, Loss: 8.4118
Epoch 400, Loss: 8.2857
Epoch 500, Loss: 8.8959
Epoch 600, Loss: 8.6075
Epoch 700, Loss: 13.9492
Epoch 800, Loss: 16.2703
Epoch 900, Loss: 22.2968
```



Що обрати?

Якщо потрібна швидка та потужна реалізація, використовуйте PyTorch.

Якщо потрібно зрозуміти алгоритм зсередини, використовуйте NumPy.